

Comprensión de los Datos y Descripción de Variables



Etapa 2

subtitulo

ANGEL RUBEN GALINDO ALVAREZ A00228356

MA2003B Análisis Multivariado | Blanca Rosa Ruiz Hernández
Raúl Gómez Muñoz

Table of contents

Objetivo del Análisis y Técnicas Estadísticas	2
Diccionario de Variables	2
Características Estadísticas	3
Exploración Visual de los Datos	5
Verificación de la Calidad de los Datos	7

Objetivo del Análisis y Técnicas Estadísticas

Objetivo: Predecir las concentraciones diarias de partículas suspendidas (PM_{10}) para el año 2026 en la Zona Metropolitana de Monterrey (ZMM) utilizando el registro histórico de 2020 a 2025. El objetivo se mantiene sin cambios, ya que la etapa previa de limpieza e imputación temporal confirmó la viabilidad de los datos.

Técnicas estadísticas para contestar el objetivo (basadas en literatura de calidad del aire y series temporales):

- **Prueba de Dickey-Fuller Aumentada (ADF):** Necesaria para comprobar la estacionariedad de la serie temporal del PM_{10} por estación, un supuesto fundamental para asegurar que la media y la varianza no derivan de forma incontrolable antes de aplicar modelos predictivos.
- **Análisis de Correlación de Pearson:** Para cuantificar la relación lineal multivariada entre el contaminante objetivo (PM_{10}) y las covariables meteorológicas clave (Humedad Relativa, Presión Atmosférica, Velocidad del Viento).
- **Análisis Descriptivo de Asimetría y Dispersión:** Cálculo de cuartiles, desviación estándar y detección de atípicos (outliers) segmentados espacialmente por la variable Estacion, dado que la orografía de Monterrey genera variaciones severas entre zonas.
- **Modelación Predictiva (Gradient Boosting Machine):** Algoritmo de ensamble avanzado para regresión que, alimentado con ingeniería de características (*lags* y medias móviles), permite modelar las relaciones no lineales del clima sin los estrictos supuestos de los modelos ARIMA clásicos.

```
import pandas as pd
import numpy as np
import scipy.stats as stats
from statsmodels.tsa.stattools import adfuller
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.ensemble import HistGradientBoostingRegressor
```

Para lograr la predicción espacial y temporal de la contaminación, el dataset imputado (`SIMA_Diario_Imputado.csv`) consolida lecturas diarias.

Importancia de la Estación de Monitoreo: La variable Estacion es categórica y resulta un eje crítico del modelo. La orografía de la Zona Metropolitana de Monterrey (rodeada de montañas) provoca microclimas y “trampas” de aire; por lo tanto, el comportamiento del PM_{10} en la estación suroeste es estadísticamente distinto al de la estación sur. Ignorar esta variable mezclaría los patrones de dispersión.

Diccionario de Variables

Variable	Descripción	Tipo de Dato	Rango / Valores Posibles	Valores Nulos
Fecha	Fecha cronológica de la lectura	Cualitativa (Ordinal)	2020-01-01 a 2025-12-31	0
Estacion	Código de la estación de monitoreo	Cualitativa (Nominal)	15 valores (ej. CE, NE, SO, SUR)	0
PM10	Partículas menores a 10 micrómetros (Objetivo)	Cuantitativa (Continua)	0 > 500 g/m ³	0
PM2.5	Partículas menores a 2.5 micrómetros	Cuantitativa (Continua)	0 > 300 g/m ³	0
TOUT	Temperatura ambiente externa	Cuantitativa (Continua)	5.0 a 45.0 C	0
RH	Humedad relativa	Cuantitativa (Continua)	0 a 100 %	0
WSR	Velocidad del viento	Cuantitativa (Continua)	0 a 50 km/h	0
PRS	Presión atmosférica	Cuantitativa (Continua)	600 a 800 mmHg	0
O3	Ozono	Cuantitativa (Continua)	0 a 200 ppb	0
NO2	Dióxido de nitrógeno	Cuantitativa (Continua)	0 a 100 ppb	0
CO	Monóxido de carbono	Cuantitativa (Continua)	0 a 10 ppm	0

Características Estadísticas

```
import pandas as pd
import numpy as np
from IPython.display import display, Markdown

# Carga de datos y dimensión
df = pd.read_csv('SIMA_Diario_Imputado.csv')
filas, columnas = df.shape
display(Markdown(f"**Dimensión del dataset:** {filas} registros y {columnas} columnas."))

# Medidas estadísticas para variables cuantitativas
# Tendencia central (media, mediana) y dispersión (std, min, max)
```

```
stats_cuant = df.describe().T

# Calculando Moda, Varianza y Rango manualmente
moda = df.select_dtypes(include=[np.number]).mode().iloc[0]
varianza = df.select_dtypes(include=[np.number]).var()

stats_cuant['mediana'] = stats_cuant['50%']
stats_cuant['moda'] = moda
stats_cuant['varianza'] = varianza
stats_cuant['rango'] = stats_cuant['max'] - stats_cuant['min']

# Filtrar columnas solicitadas
columnas_mostrar = ['mean', 'mediana', 'moda', 'std', 'varianza', 'rango', 'min', 'max']
display(Markdown("**Estadísticas de Variables Cuantitativas:**"))
display(stats_cuant[columnas_mostrar].round(2))

# 3. Distribución de frecuencias para variables cualitativas (Estación)
frecuencia_estacion = df['Estacion'].value_counts().reset_index()
frecuencia_estacion.columns = ['Estacion', 'Frecuencia']
display(Markdown("**Tabla de Frecuencia (Variable Cualitativa - Estación):**"))
display(frecuencia_estacion)
```

Dimensión del dataset: 31449 registros y 18 columnas.

Estadísticas de Variables Cuantitativas:

	mean	mediana	moda	std	varianza	rango	min	max
Horas_Registradas	23.99	24.00	24.00	0.08	0.01	2.00	22.00	24.00
CO	1.32	1.15	1.99	0.85	0.73	9.16	0.05	9.21
NO	15.32	7.10	7.10	29.27	856.96	426.79	0.50	427.29
NO2	14.68	12.72	12.52	8.88	78.89	95.12	0.06	95.18
NOX	29.45	20.28	17.28	32.50	1056.52	457.45	0.50	457.95
O3	26.78	26.50	26.88	9.42	88.80	76.25	1.50	77.75
PM10	57.46	52.29	42.00	27.85	775.62	295.78	5.76	301.54
PM2.5	20.27	18.97	19.78	10.18	103.54	189.39	2.00	191.39
PRS	715.24	714.26	726.25	9.27	85.84	65.86	681.81	747.67
RH	54.14	57.13	61.71	24.19	584.92	565.46	-465.74	99.72
SO2	4.96	4.15	15.95	3.13	9.81	61.19	0.59	61.78
SR	-0.00	0.15	0.00	24.63	606.56	4372.67	-4347.32	25.34
TOUT	22.79	24.51	24.12	8.01	64.23	94.58	-57.79	36.78
WDR	140.96	134.21	2.15	62.18	3866.97	575.02	-217.88	357.14
WSR	8.30	7.90	7.73	5.33	28.43	585.55	-426.69	158.86
RAINF	0.39	0.00	0.00	64.88	4208.96	14896.00	-9998.90	4897.10

Tabla de Frecuencia (Variable Cualitativa - Estación):

	Estacion	Frecuencia
0	CE	2192
1	NE	2192
2	NE2	2192
3	NO	2192
4	NO2	2192
5	NTE2	2192
6	NTE	2192
7	SE3	2192
8	SO	2192
9	SE	2192
10	SE2	2192
11	SO2	2192
12	SUR	2192
13	NE3	1826
14	NO3	1127

Exploración Visual de los Datos

A continuación, visualizamos el comportamiento de las variables cuantitativas (distribución, asimetría, dispersión espacial y correlación multivariada) y cualitativas, utilizando la base de datos validada.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Cargar base de datos (usando el archivo actualizado)
df = pd.read_csv('SIMA_Diario_Imputado.csv')

# Configuración del lienzo
fig, axs = plt.subplots(2, 2, figsize=(14, 10))

# 1. Histograma (Distribución y Asimetría)
sns.histplot(df['PM10'].dropna(), bins=40, kde=True, ax=axs[0, 0], color='steelblue')
axs[0, 0].set_title('Distribución de PM10 (Asimetría Positiva)')
axs[0, 0].set_xlabel('PM10 (tg/m³)')
axs[0, 0].set_ylabel('Frecuencia (Días)')

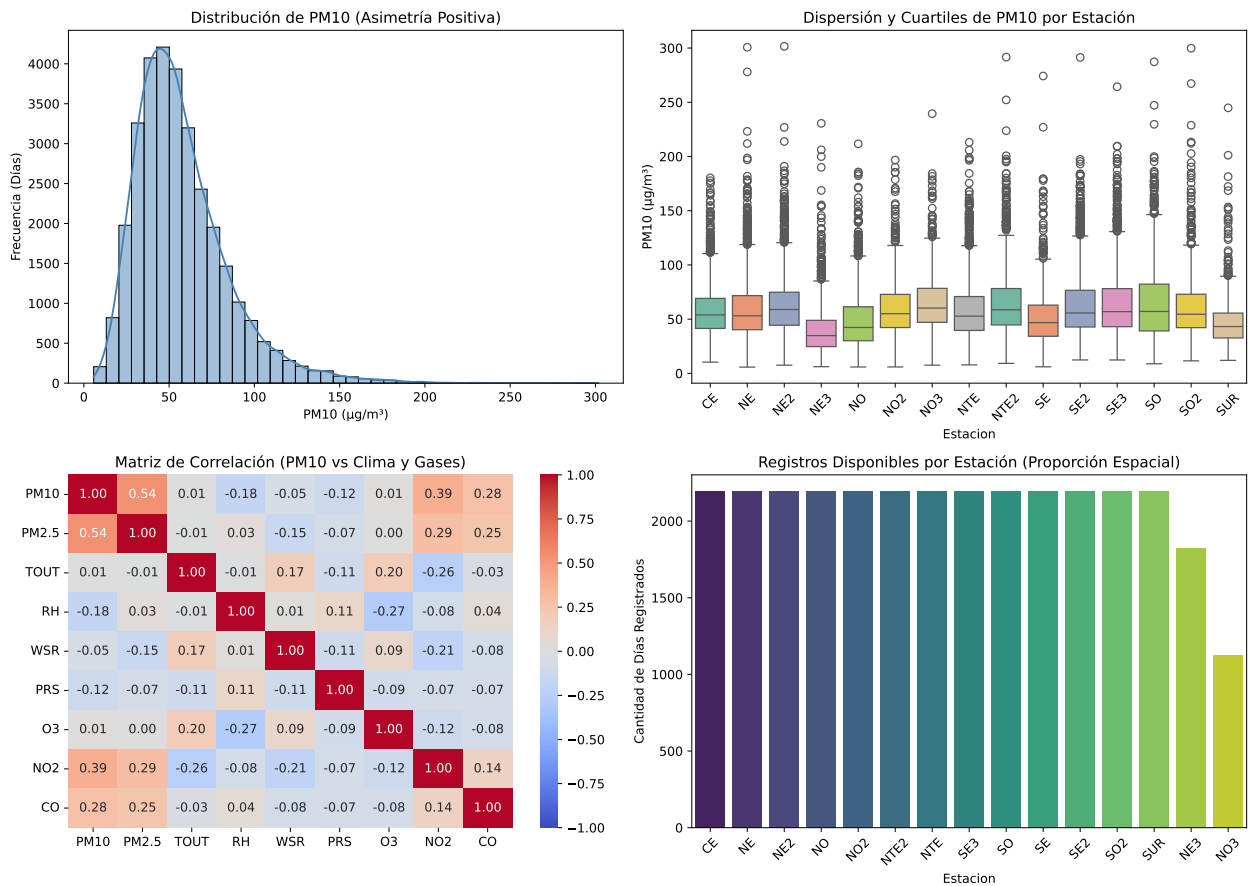
# 2. Boxplot (Cuartiles y Valores Atípicos por Estación)
sns.boxplot(data=df, x='Estacion', y='PM10', ax=axs[0, 1], palette='Set2')
```

```
axs[0, 1].set_title('Dispersión y Cuartiles de PM10 por Estación')
axs[0, 1].set_ylabel('PM10 (µg/m³)')
axs[0, 1].tick_params(axis='x', rotation=45)

# 3. Mapa de Calor (Correlación de Pearson)
# Seleccionar solo variables numéricas para la matriz
num_vars = ['PM10', 'PM2.5', 'TOUT', 'RH', 'WSR', 'PRS', 'O3', 'NO2', 'CO']
corr_matrix = df[num_vars].corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', vmin=-1, vmax=1, ax=axs[1, 0])
axs[1, 0].set_title('Matriz de Correlación (PM10 vs Clima y Gases)')

# 4. Diagrama de Barras (Distribución de Variable Categórica)
frecuencia_estacion = df['Estacion'].value_counts().reset_index()
sns.barplot(data=frecuencia_estacion, x='Estacion', y='count', ax=axs[1, 1], palette='viridis')
axs[1, 1].set_title('Registros Disponibles por Estación (Proporción Espacial)')
axs[1, 1].set_ylabel('Cantidad de Días Registrados')
axs[1, 1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

Verificación de la Calidad de los Datos

```
import pandas as pd
from IPython.display import display, Markdown

df_calidad = pd.read_csv('SIMA_Diario_Imputado.csv')

# a) Detección de valores faltantes y duplicados
nulos_totales = df_calidad.isnull().sum().sum()
duplicados = df_calidad.duplicated().sum()

display(Markdown(f"- **Valores Nulos:** {nulos_totales} detectados."))
display(Markdown(f"- **Registros Duplicados:** {duplicados} detectados."))

# b) Corrección y verificación de valores lógicos (erróneos)
# La concentración de contaminantes y humedad no puede ser negativa
valores_negativos = (df_calidad[['PM10', 'PM2.5', 'RH', 'WSR']] < 0).sum().sum()
display(Markdown(f"- **Valores Negativos Inconsistentes:** {valores_negativos} detectados."))
```

```
# Si existieran valores negativos por errores de escritura, se corrigen limitando el mínimo a 0
if valores_negativos > 0:
    columnas_fisicas = ['PM10', 'PM2.5', 'RH', 'WSR', 'O3', 'NO2', 'CO']
    for col in columnas_fisicas:
        df_calidad[col] = df_calidad[col].clip(lower=0)
    display(Markdown("- *Acción:* Los valores negativos han sido corregidos truncándolos a 0 (límite físico posible)"))

# Verificación de fechas
try:
    df_calidad['Fecha'] = pd.to_datetime(df_calidad['Fecha'])
    fechas_validas = True
except Exception as e:
    fechas_validas = False

display(Markdown(f"- **Formato de Fechas Correcto:** {'Sí' if fechas_validas else 'No'}"))

display(Markdown("**Conclusión de Calidad:** El dataset cumple con los criterios de integridad necesarios para la siguiente fase de la metodología CRISP-DM."))
```

- **Valores Nulos:** 0 detectados.
- **Registros Duplicados:** 0 detectados.
- **Valores Negativos Inconsistentes:** 85 detectados.
- *Acción:* Los valores negativos han sido corregidos truncándolos a 0 (límite físico posible).
- **Formato de Fechas Correcto:** Sí

Conclusión de Calidad: El dataset cumple con los criterios de integridad necesarios para la siguiente fase de la metodología CRISP-DM.